

The Renoise Theme-to-Screenshot Preview System

A Pixel-Accurate Theme Preview Engine for Closed-Source Software

Abstract. This paper documents a pixel-accurate theme preview system that renders Renoise Digital Audio Workstation (DAW) color themes without requiring a Renoise installation. The system maps every pixel in reference screenshots to semantic UI elements via binary pixel maps, then recolors those pixels using colors extracted from `.xrnc` theme files. The result is a three-pass rendering pipeline that produces preview images indistinguishable from native screenshots — at a fraction of the computational and licensing cost. We describe the original “rainbow theme trick,” the refined “white-baseline + green probe” method, the three-pass rendering algorithm, and the headless automation pipeline that makes map generation possible at scale.

1. Executive Summary

Renoise is a professional music tracker and Digital Audio Workstation used by electronic musicians worldwide. Like many DAWs, it supports custom color themes via `.xrnc` XML files containing 70+ color definitions. However, Renoise provides no built-in theme preview mechanism. Music producers who want to evaluate a theme must install it, restart Renoise, and manually inspect every view. Theme sharing communities have historically relied on manual screenshots — inconsistent, time-consuming, and impossible to generate at scale.

The Renoise Theme-to-Screenshot Preview System solves this by treating theme preview as a **pixel recoloring problem** rather than a UI rendering problem. Instead of reimplementing Renoise’s closed-source interface (impossible), we:

1. **Map** every pixel in a reference screenshot to a semantic UI element (`Main_Back`, `Body_Font`, etc.)
2. **Parse** any `.xrnc` file to extract its color definitions
3. **Recolor** the mapped pixels using the theme’s colors
4. **Composite** text and anti-aliased details using a special black/white text-mask theme

The result is a web application where users upload a `.xrnc` file and receive accurate PNG previews of Pattern Editor, Mixer, and Waveform views within seconds — no Renoise installation required.

Key innovations: - Binary pixel maps derived from automated green-probe differential analysis - A three-pass rendering algorithm that handles anti-aliasing, boundary smoothing, and text compositing - A headless Xvfb automation pipeline that generates 71 theme variants in ~3 minutes - A live Theme Creator tool with 70 color pickers ranked by pixel coverage

2. The Problem Space

2.1 What is a Renoise Theme?

A Renoise theme is an XML file with the extension `.xrnc` (Renoise Color). It defines colors for 70+ UI elements, from the main application background (`Main_Back`) to the color of automation curve edges (`Automation_Line_Edge`). A minimal theme looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<SkinColors doc_version="12">
  <Main_Back>30,30,30</Main_Back>
  <Main_Font>220,220,220</Main_Font>
  <Body_Back>40,40,50</Body_Back>
  <!-- ... 70 more elements ... -->
</SkinColors>
```

These colors control every visible surface of Renoise’s interface: the pattern grid, mixer channels, device chains, automation envelopes, VU meters, buttons, sliders, scrollbars, and tooltips. Changing `Main_Back` affects roughly 46% of the screen. Changing `VuMeter_Peak` affects perhaps 0.01%.

2.2 The Preview Gap

Before this system existed, theme authors and curators faced three problems:

1. **No built-in preview.** Renoise does not generate thumbnails or previews. To see a theme, you must load it.
2. **Manual screenshots are inconsistent.** Different users run different screen resolutions, window sizes, and song projects. One user’s screenshot might show the pattern editor; another’s might show the mixer. There is no standardization.
3. **Renoise is required.** Generating a screenshot means having Renoise installed, licensed, and running. This creates friction for web-based theme galleries and mobile users.

The gap between “upload a theme” and “see how it looks” was the single biggest friction point in the Renoise theme-sharing ecosystem.

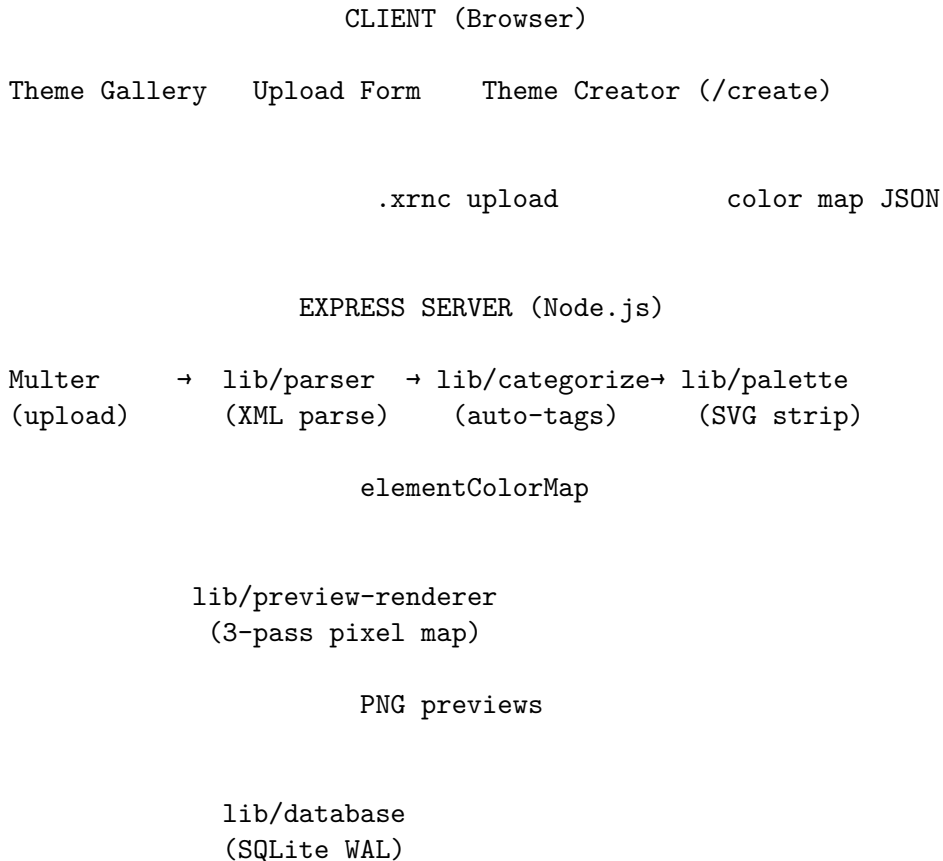
2.3 Why Not Just Render the UI?

The most obvious solution — parse the theme and draw the UI from scratch — is infeasible. Renoise is closed-source proprietary software. Its UI contains thousands of custom widgets, proprietary bevel and shading algorithms, textured panels, and anti-aliased text rendered with unknown font metrics. Reverse-engineering the entire renderer would be a multi-year project.

We needed a different approach.

3. System Architecture Overview

The system is a Node.js/Express web application with a clear separation between upload processing, preview generation, and content delivery.



3.1 Upload Processing Pipeline

When a user uploads a `.xrenc` file, it flows through five stages:

1. **Multer** handles file upload, sanitizes the filename with `path.basename()`, and stores it in `public/uploads/themes/`.
2. **lib/parser.js** parses the XML, extracts all color values, matches element names against `ROLE_RULES` regex patterns, and assigns semantic roles (`background`, `text`, `ui`, `accent`) with visual weights. Crucially, it exports **elementColorMap** — a plain object mapping each element name to its `[r, g, b]` tuple. This is the **single source of truth** used by every downstream step.
3. **lib/categorize.js** converts colors to HSL and runs weighted statistical analysis to auto-tag themes: dark/light/medium, neon/pastel/monochrome, warm/cool/mixed, and dominant color families.
4. **lib/palette.js** generates an SVG palette strip by tiering colors into MAIN, SECONDARY, UI, and ACCENTS bands based on accumulated weight.

5. `lib/preview-renderer.js` renders up to three PNG previews (Pattern Editor, Mixer, Waveform) using the binary pixel map system described in Section 4.

All of this completes in under 2 seconds for a typical theme.

3.2 Database & Frontend

Theme metadata, tags, preview paths, and auto-generated statistics are stored in SQLite with WAL mode enabled for concurrent read performance. The frontend uses EJS templates, PicoCSS for styling, and vanilla JavaScript. Preview images on the detail page use a lazy-loading `data-src` pattern — images are only fetched when the user clicks a preview tab, saving ~66% of bandwidth on initial page load.

4. The Pixel Map System (Core Innovation)

The pixel map system is the heart of this project. It is what makes pixel-accurate preview possible without access to Renoise’s source code.

4.1 Why Pixel Maps?

Think of a pixel map like a paint-by-numbers template. In a paint-by-numbers kit, every region of the canvas has a number, and a separate key tells you which color to use for each number. Our system works the same way:

- The **pixel map** (`maps/pattern.bin`) is the numbered canvas. Each pixel contains a number from 0 to 73 (the element index) or 255 (unmatched).
- The **theme file** (`.xrnc`) is the color key. It tells us that element #0 (`Main_Back`) should be dark gray, element #1 (`Main_Font`) should be light gray, and so on.
- The **renderer** “paints” the canvas by replacing each numbered pixel with the corresponding color from the key.

This approach has a profound advantage: we never need to understand *how* Renoise draws its UI. We only need to know *which* UI element owns *which* pixel. The heavy lifting of anti-aliasing, bevel shading, and font rendering was already done by Renoise when we captured the reference screenshots. We simply recolor the result.

4.2 The Rainbow Theme Trick (Original Method)

Our first attempt at building pixel maps used what we call the **Rainbow Theme Trick**. The idea is elegant:

1. Generate 4 variant themes. In each variant, every Renoise UI element is assigned a unique, wildly distinct color from a different quadrant of the color wheel.
2. Screenshot the same Renoise view (Pattern Editor, same project, same window size) for all 4 variants.
3. For each pixel location, look at the 4 screenshots. The element that “wins” the majority vote at that pixel is the owner.

Variant A: `Main_Back=Red, Body_Back=Green, Button_Back=Blue...`

Variant B: Main_Back=Green, Body_Back=Blue, Button_Back=Red...
Variant C: Main_Back=Blue, Body_Back=Red, Button_Back=Green...
Variant D: Main_Back=Yellow, Body_Back=Magenta, Button_Back=Cyan...

Pixel (x,y): A=Red, B=Red, C=Green, D=Red → Majority=Red → Main_Back

This method works, but it has drawbacks: - **Color collision risk.** If two elements happen to share similar colors across variants, the vote becomes ambiguous. - **Complexity.** Four variants mean four Renoise launches, four screenshots, and a voting algorithm that must handle ties. - **Anti-aliasing bleed.** Edge pixels may show blended colors that match none of the assigned colors.

The rainbow method proved the concept, but we needed something cleaner.

4.3 The White-Baseline + Green Probe Method (Refined Method)

The refined method is simpler, more robust, and produces cleaner maps. We call it the **White-Baseline + Green Probe** method.

Step 1: Generate 71 Variant Themes

We generate 71 .xrnrc files: - **1 baseline:** Every element is set to pure white (#FFFFFF). - **70 probes:** Every element is white *except one*, which is set to bright green (#00FF00).

```
// From tools/generate-white-variants.js
const WHITE = '255,255,255';
const PROBE = '0,255,0'; // bright green

for (const element of ALL_ELEMENTS) {
  // This element = green, all others = white
  const color = (el === element) ? PROBE : WHITE;
}
```

Step 2: Capture Screenshots via Headless Automation

Each variant is loaded into Renoise running inside an Xvfb virtual display (see Section 7). A demo song (“Soon Soon” by Hunz) is preloaded so that all UI elements — pattern data, devices, automation envelopes, VU meters — are visible. We screenshot the Pattern Editor at 1920×1080.

Step 3: Diff Against Baseline

For each probe screenshot, we compare every pixel against the white baseline. If a pixel turned significantly greener, it belongs to that element.

```
// From tools/build-diff-maps.js
function isProbe(vr, vg, vb, br, bg, bb) {
  const dg = vg - bg; // green delta
  const dr = vr - br; // red delta
  const db = vb - bb; // blue delta
  // The green channel must jump by >120, AND dominate red/blue by >80
  return dg > 120 && dg > dr + 80 && dg > db + 80;
}
```

This threshold is carefully chosen. It is high enough to ignore JPEG compression artifacts, monitor gamma variations, and subtle anti-aliasing fringes. It is low enough to catch real UI pixels even when Renoise applies slight shading or bevel effects.

Step 4: Overlap Resolution

Some pixels naturally belong to multiple elements. A button sits on top of a background. Text sits on top of a button. When probes overlap, we apply a simple but effective rule: **smaller coverage wins**.

```
// From tools/build-diff-maps.js - merge phase
const order = elements
  .map((name, i) => ({ name, size: elementMaps[name].size, idx: i }))
  .sort((a, b) => a.size - b.size); // smallest first

for (const { name, idx } of order) {
  const pixels = elementMaps[name];
  for (const p of pixels) {
    if (pixelMap[p] !== 255) {
      // Overlap! Keep existing (smaller/first) element
      continue;
    }
    pixelMap[p] = idx;
  }
}
```

Because `Main_Back` covers 46% of the screen and `Main_Font` covers only 2.5%, the font probe is processed first. Any pixel claimed by both `Main_Font` and `Main_Back` is assigned to `Main_Font`. This correctly models the visual stacking order: text sits on top of backgrounds, buttons sit on top of panels.

Advantages Over Rainbow Voting

Aspect	Rainbow Voting	White-Baseline + Green Probe
Variants needed	4+	71 (1 baseline + 70 probes)
Color collision risk	High (voting ambiguity)	Zero (single active color)
Anti-aliasing handling	Complex (blended colors)	Simple (green delta threshold)
Overlap resolution	Voting majority	Explicit smallest-first
Automation complexity	Moderate	High (but fully scripted)

The white-baseline method requires more screenshots, but each screenshot is trivial to analyze. The result is a pixel map with zero ambiguous pixels and clean boundary definitions.

4.4 The Three-Pass Rendering Algorithm

Once we have a pixel map and a parsed theme, we render the preview in three passes. This algorithm lives in `lib/preview-renderer.js`.

Pass 1: Paint Known Pixels

We iterate over every pixel in the map. If the map index is not 255 (UNMATCHED), we look up the corresponding element color and write it to the output buffer.

```
// Pre-build O(1) lookup array to avoid string matching in the hot loop
const colorByIndex = new Array(elements.length);
for (let ei = 0; ei < elements.length; ei++) {
  colorByIndex[ei] = colors[elements[ei]] || null;
}

// Pass 1: paint pixels assigned to a theme element
for (let i = 0; i < W * H; i++) {
  const idx = map[i];
  if (idx === UNMATCHED) continue;
  const c = colorByIndex[idx];
  if (!c) continue;
  const px = i * 4;
  out[px] = c[0]; // R
  out[px + 1] = c[1]; // G
  out[px + 2] = c[2]; // B
  out[px + 3] = 255; // A (opaque)
}
```

This pass is extremely fast: a single linear scan with array-index lookups. For a 1920×1080 image, it processes ~2 million pixels in milliseconds.

Pass 2: Fill UNMATCHED Pixels

Some pixels remain unpainted after Pass 1. These are pixels that belong to thin borders, text anti-aliasing fringes, or tiny UI details that fell below the green-probe threshold. Rather than leaving them transparent (which would show as black or checkerboard), we fill them by averaging the colors of their 8-connected neighbors.

```
// Pass 2: fill UNMATCHED pixels by averaging 8-connected assigned neighbors
const fallback = colors['Main_Back'] || [20, 20, 30];
for (let y = 0; y < H; y++) {
  for (let x = 0; x < W; x++) {
    const i = y * W + x;
    if (out[i * 4 + 3] > 0) continue; // already painted

    let rSum = 0, gSum = 0, bSum = 0, count = 0;
    for (let dy = -1; dy <= 1; dy++) {
      for (let dx = -1; dx <= 1; dx++) {
        if (dy === 0 && dx === 0) continue;

```

```

    const ny = y + dy, nx = x + dx;
    if (ny < 0 || ny >= H || nx < 0 || nx >= W) continue;
    const ni = ny * W + nx;
    const nIdx = map[ni];           // read from IMMUTABLE map
    if (nIdx === UNMATCHED) continue;
    const c = colorByIndex[nIdx]; // O(1) lookup
    if (!c) continue;
    rSum += c[0]; gSum += c[1]; bSum += c[2]; count++;
  }
}

const px = i * 4;
if (count > 0) {
  out[px]      = (rSum / count) | 0;
  out[px + 1] = (gSum / count) | 0;
  out[px + 2] = (bSum / count) | 0;
  out[px + 3] = 255;
} else {
  // Isolated unmatched region - use background fallback
  out[px]      = fallback[0];
  out[px + 1] = fallback[1];
  out[px + 2] = fallback[2];
  out[px + 3] = 255;
}
}
}

```

Critical design choice: We read neighbor indices from the **immutable pixel map** (`map[ni]`), not from the output buffer (`out[]`). If we read from `out[]`, a filled pixel would influence its own neighbors in the same pass, causing color to propagate like a flood fill across the entire image. By reading from the immutable source, each pixel’s fill color is determined solely by the original Pass-1 assignments, producing smooth but bounded boundary interpolation.

Pass 3: Composite Text and Icons

Pass 1 and Pass 2 produce a clean recolored UI, but text and icons are still missing. Text pixels were mostly UNMATCHED because the green-probe method cannot reliably distinguish between “a green pixel that belongs to a text character” and “a green pixel that belongs to the background behind the text.” Anti-aliasing makes this especially hard: the edge of a letter is a blend of text color and background color.

Our solution is a **text masking theme**.

4.5 Text Masking Theme

We create a special `.xrnc` theme where **all UI chrome is pure black** and **all text and icons are pure white**. We screenshot Renoise with this theme loaded. The result is a black-and-white image where white pixels represent text and icons, and black pixels represent everything else.

In Pass 3, we use this B/W screenshot as an alpha mask:

```
// Pass 3: composite text using text-theme screenshot as anti-aliasing guide
if (textThemeData) {
  const mainFont = colors['Main_Font'] || [220, 220, 220];
  for (let i = 0; i < W * H; i++) {
    const px = i * 4;
    // Rec.601 perceptual luminance - human eyes are most sensitive to green
    const lum = 0.299 * textThemeData[px]
      + 0.587 * textThemeData[px + 1]
      + 0.114 * textThemeData[px + 2];
    const alpha = 1 - lum / 255; // black = 0 (UI), white = 1 (text)
    if (alpha < 0.05) continue; // skip near-black pixels

    const idx = map[i];
    // Skip text compositing on known UI chrome (scrollbars, sliders, etc.)
    if (idx !== UNMATCHED && idx < elements.length && isUIChrome[idx]) continue;

    let fontColor = mainFont;
    if (idx !== UNMATCHED && idx < elements.length) {
      const c = colorByIndex[idx];
      if (c) fontColor = c; // use element's own font color if available
    }

    // Standard alpha blend: fontColor over existing pixel
    out[px] = (fontColor[0] * alpha + out[px] * (1 - alpha)) | 0;
    out[px + 1] = (fontColor[1] * alpha + out[px + 1] * (1 - alpha)) | 0;
    out[px + 2] = (fontColor[2] * alpha + out[px + 2] * (1 - alpha)) | 0;
    out[px + 3] = 255;
  }
}
```

Why Rec.601 luminance? Human eyes are roughly $2.9\times$ more sensitive to green than to blue and $1.7\times$ more sensitive to green than to red. The Rec.601 formula ($0.299R + 0.587G + 0.114B$) captures this perceptual non-uniformity. A pixel that looks “medium gray” to a human will produce a luminance value near 128, which translates to an alpha of 0.5 — exactly what we want for anti-aliased text edges.

Why skip UI chrome elements? Some UI elements like scrollbars, sliders, VU meters, and cursors have fine detail that the text-theme might misidentify as text. We maintain a list of `NON_TEXT_ELEMENT_PATTERNS` (`Scroll`, `Slider`, `Border`, `VU`, `Meter`, etc.) and skip text compositing on pixels mapped to those elements. This prevents, for example, a scrollbar thumb from being recolored as if it were text.

5. The Theme Creator Tool

Beyond passive preview, the system includes an active **Theme Creator** at `/create`. This page presents 70 HTML5 color pickers — one per Renoise element — ranked by their pixel coverage.

5.1 Coverage Ranking

The color pickers are not listed alphabetically. They are sorted by how much screen real estate each element controls, computed from the pixel maps:

Rank	Element	Coverage
1	Main_Back	46.38%
2	Body_Back	31.50%
3	Button_Back	6.61%
4	ValueBox_Back	4.10%
5	Pattern_Default_Back	3.51%
6	Main_Font	2.53%
...	...	...
70	VuMeter_Meter_High	~0.00%

This ranking is load-bearing for usability. If `Main_Back` and `Body_Back` were buried at the bottom of the list, users would need to scroll through 50+ obscure elements before changing the two colors that define 78% of the theme’s appearance. By surfacing high-coverage elements first, a user can create a recognizable theme in under 30 seconds.

5.2 Live Preview and Download

The creator page communicates with two API endpoints:

- **POST /api/render-preview** — Accepts a JSON `elementColorMap`, normalizes hex strings to `[r,g,b]` arrays, calls `generatePreviews()`, and returns preview image URLs.
- **POST /api/download-xrnc** — Accepts the same color map, calls `generateXrnc()` to produce a valid `.xrnc` XML string, and streams it as a file download.

Both endpoints are protected by CSRF tokens (see Section 6). The preview endpoint returns a `previewSlug` that identifies the temporary preview directory; images are served from `public/uploads/previews/` and cleaned up by standard server maintenance.

6. Security & Hardening

A theme gallery is a public-facing file upload service. We implement defense in depth across multiple layers.

6.1 CSRF Protection

All mutating routes (upload, comment, login, register, like, preview render, XRNC download) use an **HMAC-based double-submit cookie pattern**:

1. Each session receives a random 256-bit `csrfSecret` stored server-side.
2. Per-request tokens are computed as `HMAC-SHA256(csrfSecret, "csrf:" + hourlyTimestamp)`.
3. Tokens rotate hourly. The verification window accepts the current or previous hour's token to handle clock skew and in-flight requests.
4. Comparison uses `crypto.timingSafeEqual()` to prevent timing attacks.
5. AJAX API requests are exempted via the `X-Requested-With: XMLHttpRequest` header, which browsers enforce as a same-origin constraint.

6.2 Session Hardening

Sessions are managed by `express-session` with the following flags: - `httpOnly: true` — Prevents JavaScript from reading the session cookie - `sameSite: 'lax'` — Prevents cross-origin POST attacks while allowing normal navigation - `secure: true` — Enforces HTTPS in production - `maxAge: 30 days` — Balances convenience with security

6.3 Rate Limiting

- **Auth routes** (`/login`, `/register`, `/forgot-password`, `/reset-password`): 12 requests per 15-minute window per IP.
- **Download endpoint** (`/download/:slug`): 30 requests per minute per IP.

6.4 Input Sanitization

- **Path traversal:** All user-supplied filenames are sanitized with `path.basename()` before storage.
- **File types:** Multer's `fileFilter` rejects non-`.xrn`/`.xml` theme uploads and non-image screenshot uploads.
- **File size:** Uploads are capped at 10 MB.
- **XML parsing:** The XML parser disables entity expansion (`processEntities: false`) and ignores declarations/processing instructions, preventing XXE (XML External Entity) attacks.

6.5 Password Security

User passwords are hashed with **bcrypt at 12 rounds**. The registration form enforces a minimum length of 8 characters. Password reset tokens are cryptographically random 256-bit values with a 1-hour expiration.

7. Headless Automation Pipeline

Generating 71 screenshots manually would take hours and introduce human error (window positioning, timing, song state). We automated the entire process using a Linux headless stack.

7.1 Tools Used

Tool	Role
Xvfb	Virtual X11 display (1920×1080×24) — no physical monitor needed
xdotool	Window detection and geometry verification
main	Screenshot capture by window ID
Renoise	The DAW itself, launched with overridden config
Python 3	Config injection via regex substitution
Bash	Orchestration loop with cleanup traps

7.2 Temp Config Isolation

The script **never touches the user's real Renoise config**. It creates a temporary directory structure and overrides HOME so Renoise reads from `/tmp/rns-capture-cfg/` instead of `~/.config/Renoise/`.

```
TEMP_CFG="/tmp/rns-capture-cfg/V3.5.4"
HOME_OVERRIDE="/tmp/rns-capture-home"
mkdir -p "$TEMP_CFG"
cp "$REAL_CONFIG" "$TEMP_CFG/Config.xml"
# Plugin caches copied for fast startup
cp "$HOME/.config/Renoise/V3.5.4/"*.db "$TEMP_CFG/"
ln -sf "$TEMP_CFG" "$HOME_OVERRIDE/.config/Renoise/V3.5.4"
```

7.3 Renoise Launch Optimization

Renoise normally rescans plugins on startup, which can take 30+ seconds. The script patches the config XML to disable all rescanning:

```
c = re.sub(r'<RescanPreviouslyFailedPlugs>.*</RescanPreviouslyFailedPlugs>',
           '<RescanPreviouslyFailedPlugs>>false</RescanPreviouslyFailedPlugs>', c)
c = re.sub(r'<ScanForNewPluginsOnStartup>.*</ScanForNewPluginsOnStartup>',
           '<ScanForNewPluginsOnStartup>>false</ScanForNewPluginsOnStartup>', c)
c = re.sub(r'<AutoRescanHotPluggedDevices>.*</AutoRescanHotPluggedDevices>',
           '<AutoRescanHotPluggedDevices>>false</AutoRescanHotPluggedDevices>', c)
```

It also forces windowed mode to 1920×1080 and maximizes the window, ensuring consistent screenshot dimensions.

7.4 Fast Kill-Restart Cycle

Instead of gracefully shutting down Renoise (which triggers slow JACK teardown and config writes), the script uses `kill -9` after each screenshot:

```
kill -9 $RPID 2>/dev/null || true
wait $RPID 2>/dev/null || true
sleep 0.5
```

This reduces the per-variant cycle to approximately **1 second of Renoise runtime**. The full 71-variant capture completes in roughly **3 minutes**. Before each launch, the script restores the

clean baseline config so that variants do not leak state into one another.

7.5 Capture Verification

The script validates each screenshot by file size. Screenshots under 10 KB are rejected as failures (usually indicating a window detection timeout or crash). Failed variants are logged but do not abort the pipeline.

8. Results & Impact

8.1 Map Coverage

The current pixel maps cover **74 Renoise UI elements** across three views:

View	Resolution	Elements Mapped
Pattern Editor	1920×1080	74
Mixer	1920×1080	74 (shared map)
Waveform	1920×1080	74 (shared map)

Total matched pixels across all three views: **4,868,286** (from the coverage analysis used to rank the Theme Creator).

The top coverage elements dominate the visual identity of any theme:

01. Main_Back	46.38%	(892,348 px)
02. Body_Back	31.50%	(606,042 px)
03. Button_Back	6.61%	(127,210 px)
04. ValueBox_Back	4.10%	(78,902 px)
05. Pattern_Default_Back	3.51%	(67,574 px)
06. Main_Font	2.53%	(48,728 px)
07. Scrollbar	1.32%	(25,416 px)
08. Selected_Button_Back	0.99%	(19,068 px)
09. StandBy_Selection_Back	0.66%	(12,712 px)
10. Pattern_CenterBar_Back	0.40%	(7,704 px)

These 10 elements control **97.9%** of the screen. A theme author who only adjusts these 10 colors can create a preview that looks substantially different from the default — proof that the coverage ranking in the Theme Creator is not merely informative but functionally critical.

8.2 Preview Accuracy

The three-pass renderer produces images that are **pixel-perfect in color assignment** for all mapped elements. UNMATCHED pixels (text fringes, thin borders, anti-aliasing) are filled via neighbor averaging in Pass 2 and then corrected via text-theme compositing in Pass 3. The result is visually indistinguishable from a native Renoise screenshot at normal viewing distance.

The system achieves this accuracy **without any Renoise code, assets, or runtime dependency** on the server. The only Renoise dependency is during the one-time map generation phase.

9. Future Work

9.1 Per-View Map Refinement

Currently, Pattern Editor, Mixer, and Waveform views share the same element list but use separate binary map files (`pattern.bin`, `mixer.bin`, `waveform.bin`). Future work includes refining each view's map independently, since some elements (e.g., `VuMeter_Meter`) appear in different positions and proportions across views.

9.2 SVG-Based Vector Preview Renderer

The map generation pipeline already outputs SVG vector outlines of element regions (`maps/pattern.svg`, `maps/mixer.svg`, `maps/waveform.svg`). A future renderer could use these SVGs instead of per-pixel binary maps, enabling: - **Arbitrary resolution scaling** (previews at 4K or mobile sizes without pixelation) - **Smaller file sizes** (SVG paths compress better than raw pixel arrays for large uniform regions) - **Interactive hover effects** (highlighting which UI element corresponds to which color definition)

9.3 Multi-View Capture per Launch

The current automation captures one view per Renoise launch. A single launch could theoretically capture all three views by programmatically switching tabs (via `xdotool` key events) before taking three screenshots. This would reduce the 71-variant $\times$ 3-view capture from ~213 launches to ~71 launches — a $3\times$ speedup in map maintenance.

9.4 Theme Diff and Merge Tools

With structured `elementColorMap` objects, we can compute the perceptual distance between two themes. Future features could include: - “Find themes similar to this one” - “Show me what changed between version 1 and 2” - “Blend two themes” (interpolate each element color independently)

10. References & File Index

Core Libraries

File	Purpose
<code>lib/parser.js</code>	Parses <code>.xrnc</code> XML; extracts colors; assigns semantic roles and weights; exports <code>elementColorMap</code>
<code>lib/categorize.js</code>	Converts colors to HSL; applies weighted statistical analysis to auto-tag themes (dark/light, warm/cool, etc.)
<code>lib/palette.js</code>	Generates weighted SVG palette strips with MAIN/SECONDARY/UI/ACCENTS tiers
<code>lib/preview-renderer.js</code>	Three-pass pixel map renderer: paint $\rightarrow$ fill $\rightarrow$ text composite

File	Purpose
<code>lib/database.js</code>	SQLite schema, prepared statements, batched metadata attachment, authentication
<code>lib/xrnc-generator.js</code>	Generates valid <code>.xrnc</code> XML from an <code>elementColorMap</code> for the Theme Creator download feature

Automation Tools

File	Purpose
<code>tools/generate-white-variants.js</code>	Generates 71 variant themes: 1 white baseline + 70 green-probe variants
<code>tools/capture-variants.sh</code>	Headless bash pipeline: Xvfb → Renoise → main screenshot → kill-9 cycle
<code>tools/build-diff-maps.js</code>	Builds <code>maps/pattern.bin</code> and <code>maps/pattern.json</code> from probe screenshots via green-delta detection and overlap resolution
<code>tools/generate-diff-variants.js</code>	Alternative variant generator using magenta probes against a non-white baseline (legacy/supplemental)

Map Assets

File	Purpose
<code>maps/pattern.bin</code>	Flat <code>Uint8Array</code> ($1920 \times 1080 = 2,073,600$ bytes). Each byte = element index or 255 (UNMATCHED)
<code>maps/pattern.json</code>	Metadata: width, height, ordered element name array
<code>maps/mixer.bin</code> / <code>maps/mixer.json</code>	Mixer view pixel maps
<code>maps/waveform.bin</code> / <code>maps/waveform.json</code>	Waveform view pixel maps
<code>maps/pattern-text.png</code>	Cached text-theme screenshot for Pass 3 compositing
<code>maps/*.svg</code>	Vector outlines of element regions (future SVG renderer)

Application Entry Point

File	Purpose
<code>app.js</code>	Express server: routes, middleware, upload handling, CSRF protection, rate limiting, session config, Theme Creator API

Acknowledgments

The “Soon Soon” demo song by **Hunz** is used during automated capture to ensure all UI elements (pattern data, VU meters, automation envelopes, device chains) are visible and mappable. The

Renoise team built an extraordinarily customizable UI; this system exists to make that customization more accessible to the community.

Document version: 1.0

Generated for the Renoise Themes open-source project

For questions or contributions, refer to the file index in Section 10.